Universidad Sergio Arboleda

Escuela de Ingeniería y Ciencias Exactas

Programa de Ciencias de la Computación e Inteligencia Artificial

Sistema de Reservas Aéreas

Panel gerencial de ocupación: EMR, Glue y Athena

Segundo Examen Parcial

Big Data and Data Engineering

Autores

Julian Cristancho

Andrés Urrego

Semilla (seed) del equipo: 2682

Bogotá D.C., Colombia

22 de octubre de 2026

Índice

1. Resumen **2**

2. Arquitectura **2**

3. Decisiones de diseño **3**
3.1. Modelo de hosting 3
3.2. Por qué EMR 3
3.3. Layout de los datos 4
3.4. Cómo obtiene el panel sus números 5
3.5. Separación de cargas 6

4. Justificación de servicios **6**

5. Proyección de costos **6**

6. Resultados del harness **8**

7. Trazabilidad con los requisitos **9**

8. Riesgos y mejoras pendientes **9**

A. Bitácora de prompts **10**

1 Resumen

Este documento justifica las decisiones de arquitectura del segundo parcial. Al sistema de reservas del parcial 1 se le agregó un frontend, un trabajo por lotes en EMR que limpia dos años de historia (2.000.542 reservas curadas) y una capa de consulta con Glue Data Catalog y Athena. Con eso el gerente ve la ocupación por ruta y mes en un panel.

El harness oficial dio 70/70 puntos en la corrida que se incluye en la sección 6. La proyección de costo declarada es de US$25,49 al mes, bajo el tope de US$50.

Hay tres cosas que conviene declarar desde el principio:

- **Campo `bookings`.** El contrato textual dice que `bookings` cuenta solo reservas CONFIRMED, pero el oráculo del harness (`oracle.py`) cuenta CONFIRMED más CANCELLED. Seguimos al contrato escrito. Ningún test revisa ese campo.
- **Servicio Lambda en R11.** El harness detectó 7 servicios facturables: Athena, EC2, EMR, Glue, Lambda, RDS y S3. Las funciones Lambda (RedshiftOverwatch, MainMonitoringFunction, ModLabRole, RoleCreationFunction, RedshiftEventSubscription) pertenecen al propio Learner Lab. Nosotros no creamos ninguna.
- **Lugar donde se corrió el harness.** Se corrió desde la propia EC2 con URLs `localhost`, porque la EC2 no alcanza su propia IP pública (el grupo de seguridad solo permite IPs del equipo). El `config.yaml` entregado usa la IP pública fija.

2 Arquitectura

La figura 1 muestra lo que está desplegado en la cuenta del Learner Lab (us-east-1). Hay dos caminos que no se tocan entre sí. El transaccional va de la EC2 a RDS. El analítico va de S3 a EMR, de EMR a S3 curado, y de ahí a Athena, que la API consulta solo para el panel.

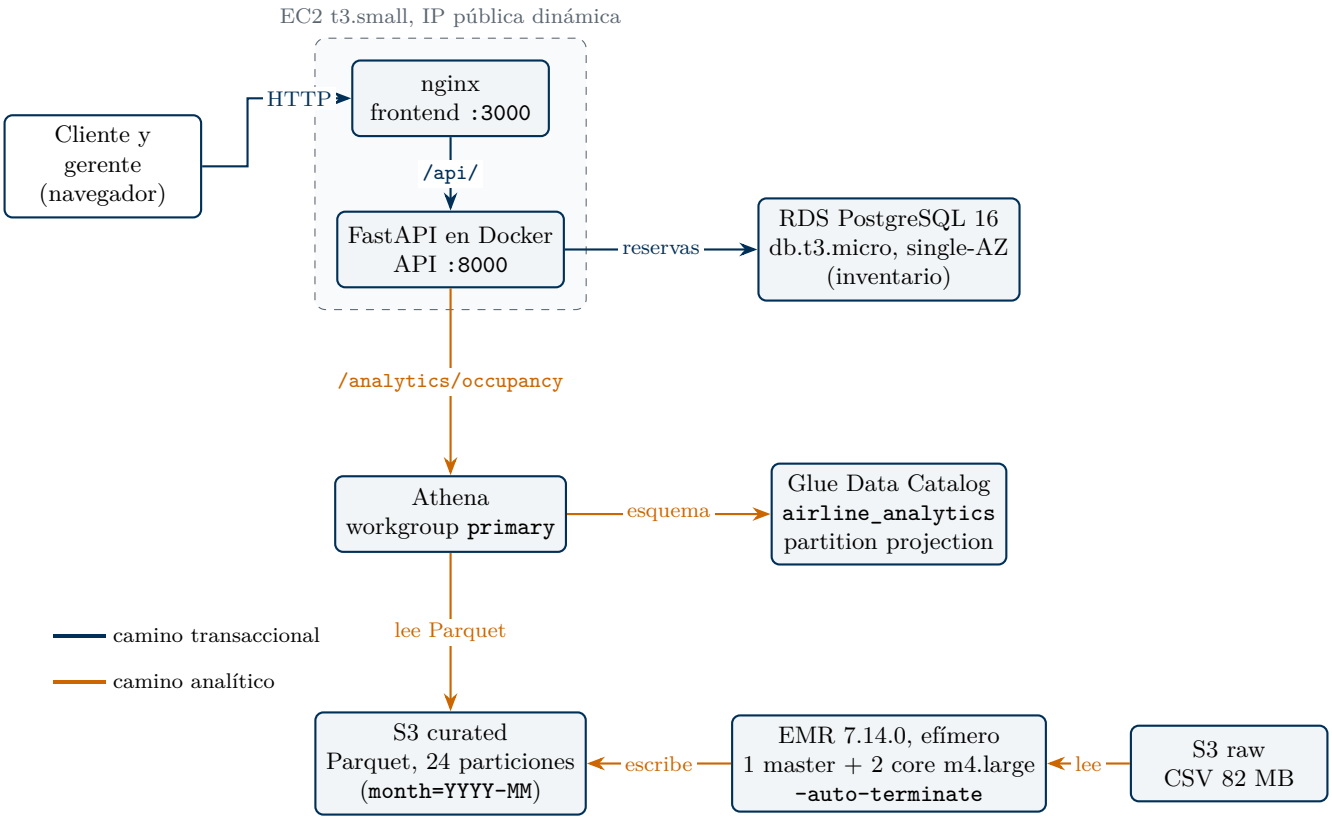


Figura 1: Arquitectura desplegada. Todo está en una VPC por defecto, una sola AZ, sin NAT ni balanceador.

Flujos. El cliente busca y reserva en `/` y la API actualiza el inventario en RDS. El gerente abre `/manager`. nginx reenvía `/api/` a FastAPI, que lanza una consulta parametrizada a Athena sobre `route_month_metrics` y devuelve el JSON. El panel solo muestra ese número. EMR no está en el camino de las peticiones: corre cuando se necesita refrescar los datos, lee `raw/` y reescribe `curated/`. Los parámetros de la base y las credenciales están en SSM Parameter Store (`/airline/db/*`), y la EC2 usa el rol del laboratorio para llamar a Athena, sin claves en el código.

3 Decisiones de diseño

3.1 Modelo de hosting

Requisitos: R1, R2, R11, R12 y RF-14 del parcial 1. *Pilares: eficiencia de costo, fiabilidad, eficiencia de rendimiento*

Backend y frontend corren como dos contenedores Docker en una sola EC2 t3.small (`airline-api` en el 8000 y `airline-web`, nginx, en el 3000). El inventario está en RDS PostgreSQL 16 db.t3.micro, single-AZ. Se eligió así porque es lo más barato que cumple R1 y R2. La reserva es una transacción con un `UPDATE` condicional (`seats_available = seats_available - 1 WHERE seats_available > 0`), y eso lo resuelve PostgreSQL sin servicios extra. En la corrida del harness la búsqueda dio p95 de 0,084 s con 20 clientes simultáneos (límite 1,0 s) y las 5 rondas de 50 reservas simultáneas dejaron exactamente 1 respuesta 201 y 49 respuestas 409.

Se descartaron alternativas más gestionadas (por ejemplo contenedores en un servicio orquestado detrás de un balanceador) porque el balanceador solo ya cuesta unos US$16 al mes y R11 pide una arquitectura simple. Se descartó instalar PostgreSQL dentro de la EC2 para no encargarnos nosotros de copias de seguridad y parches, que RDS ya incluye. Multi-AZ está prohibido por R11, así que aceptamos un riesgo de fiabilidad: si la zona cae, el sistema cae. Para este parcial es un riesgo aceptable y está declarado.

El frontend estático va en la misma EC2 y no en S3 con CloudFront. Así no se agrega un servicio facturable y el navegador habla con un solo origen, lo que evita problemas de CORS. Son tres páginas: la del cliente (buscar, elegir tarifa, reservar y ver el código en un pase de abordar), la de *check-in* y el panel gerencial (ocupación de la ruta en el mes, un plano de cabina donde cada silla es el 1 % y barras con los 12 meses del año de esa ruta). No usamos IP elástica para no pagar US$3,65 al mes; la IP pública cambia en cada arranque y `infra/encender.sh` la escribe en `config.yaml`.

Seguridad: la base nunca se abre a internet. La RDS no es accesible públicamente: no tiene IP pública y solo se llega a ella desde la VPC (la API y Glue). Los puertos 8000 y 3000 del grupo de seguridad de la API solo permiten las IPs del equipo. Para la ventana de calificación, el script `infra/abrir_calificacion.sh` los abre a internet y `infra/cerrar_calificacion.sh` los cierra después. Es temporal porque `POST /admin/flights`, que exige el contrato, no tiene autenticación.

Lo que se agregó del parcial 1. El sistema ahora implementa las clases tarifarias y el *check-in* que definimos en el parcial 1 (RN-08, RF-20 a RF-24, RF-81 y RF-82). Hay cinco clases: XS, S y M eligen silla solo en el *check-in*; L y PR la pueden elegir al comprar. El *check-in* abre 24 horas antes y cierra 1 hora antes en vuelos nacionales o 2 horas antes en internacionales. Hay 32 rutas, 4 de ellas internacionales. Mientras el pasajero escoge, la silla queda bloqueada 5 minutos, pero lo que de verdad impide que dos pasajeros tengan la misma silla es la clave primaria `(flight_id, seat_no)` de `seat_assignments`: el segundo intento recibe 409. Para no romper el contrato del harness, `ECONOMY` sigue funcionando como alias de M y la tarifa por defecto del frontend es M, que no pide silla. Las pruebas `tests/test_checkin.py` (20 bloqueos simultáneos de la misma silla, solo uno gana) pasan junto con las del parcial 1.

3.2 Por qué EMR

Requisitos: R4, R10, R11, R12. *Pilares: eficiencia de costo, excelencia operativa, sostenibilidad*

El trabajo por lotes (`emr/curate.py`, PySpark) lee 2.000.000 de reservas sucias y sus vuelos y aplica C1 a C5. Quita espacios y pasa a mayúsculas la ruta y el estado, descarta estados distintos de CONFIRMED y CANCELLED, descarta precios nulos o menores o iguales a cero, y elimina `reservation_id` duplicados. Luego une con los vuelos limpios, escribe el detalle en Parquet particionado por mes y calcula la tabla `route_month_metrics` (ocupación, ingresos, reservas y cancelaciones por ruta y mes). La capacidad se suma desde los vuelos y no desde las reservas, para que un vuelo sin ventas siga contando sus asientos. Todo se lee como texto y los tipos se convierten a mano, porque `price_cents` llega vacío o con decimales y Spark lo infería mal.

Regla	Definición	Cómo se aplica en `curate.py`
C1	Ruta sin espacios y en mayúsculas	`upper(regexp_replace(route, '\s+', ''))` sobre los vuelos
C2	Estado sin espacios y en mayúsculas	`upper(trim(status))`
C3	Solo CONFIRMED o CANCELLED	Filtro `status IN (...)`
C4	Precio presente y mayor que 0	Cast a `double`, filtro, cast final a `bigint`
C5	Quitar `reservation_id` duplicados	`dropDuplicates` sobre `reservation_id`

Tabla 1: Reglas de limpieza y su implementación. Los duplicados son copias exactas, por eso no importa cuál se conserve.

Lo que hace el lote y Athena sola no hace es dejar los datos en el formato correcto. Athena consulta, pero sobre el CSV sucio tendría que repetir la limpieza en cada consulta y leer los 82 MB cada vez. El lote limpia una vez y deja Parquet particionado, de modo que las consultas del panel son baratas.

Alternativas consideradas:

- **Athena CTAS.** Podría expresar C1 a C5 en SQL y escribir Parquet particionado. No la usamos porque R4 exige que el Parquet lo escriba una corrida de EMR, y porque con Spark controlamos el número de archivos por partición.
- **Job de Glue.** También es Spark y cobra por DPU-segundo sin administrar clúster. Tampoco cumple R4, que pide EMR. Lo usamos en el parcial 1 para el ETL de OLTP a OLAP, no aquí.

Elegimos un clúster EMR y no EMR Serverless (no probamos Serverless). Es un lote corto y un clúster efímero da el mismo resultado, con un control directo del apagado y del conteo de horas normalizadas que R10 mide. El clúster es emr-7.14.0 con 1 master y 2 core, todos `m4.large`, que es el único tipo permitido por R11. Son los nodos mínimos para que Spark distribuya el trabajo.

El apagado (R10) se garantiza de cuatro formas. El clúster se crea con `-auto-terminate`, así que se apaga cuando termina el último paso. El paso tiene `ActionOnFailure=TERMINATE_CLUSTER`, así que también se apaga si falla. El script `emr/run_emr.sh` espera con `aws emr wait cluster-terminated`. Al final lista `aws emr list-clusters -active`, que salió vacío.

Se hicieron dos corridas, cada una de 12 horas normalizadas: primero una muestra del 5 % para validar el job sin gastar presupuesto (`j-060059464906OD8KSHE`) y después la corrida completa (`j-06661302ZS6EJJ3Z0HBI`). Suman 24 horas normalizadas, contra un límite de 150. La corrida completa fue la última y sobrescribió `curated/`, así que los 24 archivos Parquet se escribieron dentro de una ventana COMPLETED, como pide R4.

3.3 Layout de los datos

Requisitos: R5, R6, R12. *Pilares: eficiencia de rendimiento, eficiencia de costo, sostenibilidad*

Por qué Parquet. Es columnar y comprimido. Athena cobra por bytes leídos y solo lee las columnas que la consulta usa. El CSV crudo pesa 82 MB y el detalle en Parquet pesa 19,5 MB (19.478.722 B en 24 archivos), un 76 % menos.

Por qué partición por mes. Las consultas del panel filtran por mes. Con `month=YYYY-MM` Athena abre solo esa carpeta. En la verificación, una consulta de un mes (`count` y `sum(price_cents)` con `WHERE month='2024-01'`) escaneó 308.482 B. El tamaño promedio de una partición es 811.613 B, así que el límite de R6 (dos veces el promedio) es 1.623.226 B y quedamos en 0,38 veces el promedio. Antes de escribir se hace `repartition("month")`, así queda un archivo por mes y no muchos archivos pequeños.

Qué pasaría sin partición. La consulta tendría que recorrer la columna de precios de los 24 meses. Estimamos unos 24 por 308 KB, es decir unos 7,4 MB, casi cinco veces el límite de 1,62 MB, y R6 fallaría. Si fuera `SELECT *` serían los 19,5 MB, y sobre el CSV, 82 MB. Esta cifra es una estimación a partir de lo medido por mes; no se ejecutó esa consulta.

Ruta en S3 (bucket del proyecto)	Contenido
`raw/`	`flights.csv` (0,4 MB) y `reservations.csv` (82 MB), sucios
`raw-sample/`	Todos los vuelos y 1 de cada 20 reservas (5 %), para probar
`curated/reservations_curated/ month=YYYY-MM/`	Detalle limpio en Parquet, 24 particiones, 19,5 MB en total
`curated/route_month_metrics/`	Métricas por ruta y mes en un solo archivo Parquet (unos 9 KB)
`athena-results/`, `emr/logs/`	Resultados de Athena y registros de EMR

Tabla 2: Organización de los datos.

Catálogo. Las dos tablas están en la base `airline_analytics` de Glue, con columnas en el orden del contrato. `reservations_curated` usa *partition projection* (`projection.month.type=date`, formato `yyyy-MM`, rango 2023-01 a 2024-12). Athena calcula las rutas de las particiones sin consultar el catálogo, no hay que correr `MSCK REPAIR TABLE` y el catálogo no se desactualiza. La limitación es que, si llegan meses fuera del rango, hay que ampliarlo.

3.4 Cómo obtiene el panel sus números

Requisitos: R6, R7, R9, R12. *Pilares: eficiencia de rendimiento, eficiencia de costo, excelencia operativa*

El panel llama a `GET /analytics/occupancy` y el endpoint consulta Athena. No consulta RDS ni un archivo local, de modo que el número que ve el gerente es el que sale del lago de datos que produjo EMR. El panel solo formatea `occupancy` en porcentaje con tres decimales (por ejemplo 60,960 %), así que coincide con el endpoint dentro de la tolerancia de R9.

La consulta es `WHERE route = ? AND month = ?` con `ExecutionParameters`, y además la ruta y el mes se validan con expresiones regulares antes de llamar a Athena (422 si el formato es malo). Así nunca se concatena texto del usuario al SQL. Hay 404 si no hay fila y 503 si Athena falla, con un tiempo máximo de 9 s para respetar los 10 s de R7.

- **Latencia.** En el harness, las 5 consultas de prueba de R7 tardaron entre 0,80 y 1,41 s. Es más lento que una consulta a RDS, pero aceptable para un panel gerencial.
- **Costo por consulta.** Athena cobra US$5 por TB y mínimo 10 MB por consulta, es decir unos US$0,00005. La tabla de métricas pesa kilobytes (el top 10 por ocupación escaneó 1.318 B), así que cada consulta paga el mínimo.
- **Frescura.** Los datos son tan frescos como la última corrida de EMR. Es un sistema por lotes: un reporte gerencial histórico no necesita datos de hoy.
- **Caché.** Sí se justifica, con una condición. La primera consulta de cada par (ruta, mes) siempre va a Athena, que es lo que R7 verifica. Después se guarda en memoria durante 10 minutos, solo para

respuestas exitosas, porque el dato solo cambia cuando corre EMR. El caché es por proceso (la API tiene 2 workers) y es suficiente aquí. Con más usuarios haría falta un caché compartido.

3.5 Separación de cargas

Requisitos: R1, R2 y RF-14 del parcial 1. *Pilares: fiabilidad, eficiencia de rendimiento*

Las búsquedas y reservas usan solo RDS. El panel usa solo Athena y S3. Ninguna consulta analítica toca la base transaccional, así que un gerente consultando no consume conexiones ni CPU del inventario. El trabajo pesado de limpieza corre en el clúster EMR, que lee y escribe en S3 y no tiene conexión a RDS. Si Athena falla, solo `/analytics/occupancy` responde 503 y el resto de la API sigue funcionando. La evidencia es que R1 y R2 se midieron con el sistema completo desplegado.

Una limitación honesta: la API de reservas y el endpoint analítico comparten la misma EC2 y sus 2 workers. Un pico de consultas al panel podría competir por CPU de esa máquina aunque no toque la base. Con el caché y el costo en CPU de una llamada a Athena (casi todo es espera de red) lo consideramos bajo, pero no lo medimos bajo carga mixta.

4 Justificación de servicios

Servicio	Alternativa considerada	Requisito	Pilares
EC2 t3.small	Contenedores orquestados con balanceador	R1, R2, R11, R12	Costo, rendimiento
RDS PostgreSQL 16 db.t3.micro, single-AZ	PostgreSQL en la EC2; Multi-AZ	R2, R11, RF-14	Fiabilidad, costo, seguridad operativa
S3 (raw, curated, resultados de Athena)	Datos en RDS o en disco de la EC2	R4, R6	Costo, sostenibilidad, durabilidad
EMR (clúster efímero, m4.large)	Glue job; Athena CTAS; EMR Serverless	R4, R10, R11	Costo, excelencia operativa
Glue Data Catalog	Crawler o `MSCK REPAIR TABLE`	R5	Excelencia operativa, costo
Athena (workgroup `primary`)	Redshift; consultar RDS	R6, R7, R9	Costo (pago por consulta), rendimiento
SSM Parameter Store	Variables en el código o en la imagen	Parcial 1 (credenciales)	Seguridad
nginx (en la EC2)	S3 con CloudFront	R8, R9, R11	Costo, simplicidad
IP pública dinámica (sin Elastic IP)	Elastic IP	Operación de la calificación	Costo

Tabla 3: Servicios, alternativas y trazabilidad. Redshift, Serverless y el balanceador se descartaron por costo o por no ser necesarios; no se desplegaron para compararlos.

5 Proyección de costos

Requisitos: R11, R12. *Pilares: optimización de costos, sostenibilidad*

Supuestos (de `cost_projection.yaml`). Despliegue 24/7 con 730 horas al mes en us-east-1, precios on-demand de `pricing.yaml`. Unos 50 usuarios, unas 2.000 peticiones al día a la API y unas 50 consultas al día al panel. Datos: unos 100 MB de CSV y unos 20 MB de Parquet. Una corrida de EMR al mes, con 3 nodos m4.large durante media hora. Sin NAT, balanceador ni Multi-AZ.

Servicio	**Detalle**	**US$/mes**
RDS	db.t3.micro single-AZ, 730 h × 0,017 = 12,41, más 20 GB gp3 = 2,30	14,71
EC2	t3.small reservada 1 año sin pago inicial, 730 h × 0,013 = 9,49, más EBS gp3 de 8 GB = 0,64	10,13
S3	Unos 120 MB (raw, Parquet, resultados de Athena) y solicitudes	0,10
Athena	Unas 3.000 consultas al mes, mínimo 10 MB cada una (unos 30 GB facturables)	0,15
EMR	1 corrida al mes, 3 × m4.large, 12 horas normalizadas × 0,03, más arranque	0,40
Glue	Data Catalog dentro de la capa gratuita	0,00
	Total declarado	**25,49**

Tabla 4: Líneas de `cost_projection.yaml`. El archivo no trae total, el harness lo calcula.

El total de US$25,49 está bajo el tope de US$50. La EC2 se proyecta como instancia reservada de 1 año sin pago inicial porque corre las 24 horas; el ahorro frente a bajo demanda es de unos US$5,70 al mes. En el Learner Lab no se pueden comprar reservas, así que lo desplegado corre bajo demanda. No usamos Elastic IP para ahorrar sus US$3,65 al mes; a cambio la IP cambia en cada arranque y `encender.sh` actualiza `config.yaml`. RDS y EC2 suman casi todo el costo (US$24,84 de 25,49) y no dependen del volumen de datos analíticos, porque el panel no pasa por ellos. Ahorramos al no usar balanceador (unos US$16 al mes), NAT ni Multi-AZ, al pagar EMR solo durante la corrida y al quitarle la IP pública a la RDS (AWS cobra US$3,65 al mes por cada IPv4 pública).

Qué pasa con ×10 en datos. Con 20 M de filas el Parquet sería de unos 200 MB y el S3 seguiría costando centavos (menos de US$1). La partición por mes mantiene lo que Athena escanea por consulta de un mes en unos 3 MB, que sigue cerca del mínimo facturable de 10 MB. Lo que sí crece es EMR: si el tiempo de la corrida creciera de forma lineal, pasaría de 0,5 h a unas 5 h y el costo de unos US$0,40 a unos US$4. Con eso el total subiría a unos US$29, todavía bajo US$50. El tope de 150 horas normalizadas de R10 se sobrepasaría con una corrida de más de unas 12 horas con el mismo clúster, así que habría que ampliar los nodos o procesar por incrementos mensuales. Estas cifras son estimaciones del equipo, no mediciones.

Qué pasa con ×10 en consultas. Serían unas 30.000 consultas al mes. Con el mínimo de 10 MB, unos 300 GB facturables, Athena costaría unos US$1,50 al mes (hoy 0,15). El caché de 10 minutos reduce este valor porque las consultas repetidas del mismo par no vuelven a Athena. Las 20.000 peticiones al día a la API siguen en una t3.small si mantienen la latencia que medimos (p95 de 0,077 s con 20 clientes), pero eso no está probado a ese volumen. El total subiría a unos US$27.

Sostenibilidad. Menos bytes leídos es menos cómputo y menos energía. Parquet más partición reducen los bytes escaneados, el clúster de EMR existe solo 10 a 15 minutos por corrida en vez de estar siempre encendido, y no hay recursos inactivos duplicados (NAT, segunda zona). Cuando el volumen crezca, el ahorro aumenta, porque el costo analítico depende de los bytes que se lean y no del tamaño total del lago.

6 Resultados del harness

El `report.md` final (copia en `evidence/p2/report.md`) da **70,0 de 70,0** puntos. Todos los requisitos pasaron.

Req.	Criterio	Puntos	Evidencia del reporte
R1	Latencia de búsqueda	4/4	100 peticiones, 20 clientes: p50 0,043 s, p95 0,084 s
R2	Sin sobreventa	10/10	5 rondas de 50: 1 respuesta 201 y 49 respuestas 409; con K=3: 3 respuestas 201; sin 5xx
R3	Consulta de reserva	3/3	200 si existe, 404 si no
R4	EMR produce Parquet	8/8	3 corridas COMPLETED; 24 de 24 archivos escritos dentro de una corrida; 2.000.542 filas, igual que el oráculo
R5	Tablas en Glue	6/6	Contrato de ambas tablas y *partition projection* activa
R6	Correctitud y poda	8/8	Top 10 igual al oráculo; consulta de un mes escaneó 308.482 B, límite 1.623.227 B
R7	Endpoint con Athena	8/8	5 pares entre 1,3 y 1,5 s; 8 consultas en el historial de Athena
R8	Flujo del cliente	6/6	La interfaz creó la reserva EPWVY4 y el backend la devolvió
R9	Panel igual al endpoint	4/4	CLO-MDE 2024-08: panel, endpoint y valor esperado 0,6096
R10	Sin EMR encendido	5/5	0 clústeres activos; 36,0 horas normalizadas (tope 150)
R11	Arquitectura simple	4/4	7 servicios facturables; instancias permitidas; sin NAT ni Multi-AZ
R12	Proyección de costos	4/4	Declarado 25,49; estimado en vivo 28,67 (bajo demanda)

Tabla 5: Resultado por requisito.

Además de la corrida del harness, verificamos Athena contra el oráculo de la semilla 2682 (archivo `verificacion_analytics.txt` en `evidence/p2/`). Coincidieron exactamente el conteo de filas (2.000.542), el ingreso CONFIRMED de 2023-01 (2.064.557.404), 2024-01 (2.072.070.154) y 2024-12 (1.983.343.325), y el conteo de 2024-01 (83.506). El top 10 por ocupación coincide en orden y valor, con CUC-BAQ (0,8792) y CUC-BGA (0,8529) a la cabeza.

Pruebas con otras sondas. El profesor corre el harness con la misma semilla pero con otros pares de ruta y mes en R7 y R9. Para simularlo cambiamos `probe_salt` en una copia del harness (82682, 1234 y 777) y corrimos las pruebas de analítica y frontend. Pasaron las 12 pruebas con cada valor; la consulta más lenta del endpoint tardó 1,35 s. El detalle está en `evidence/p2/sondas_distintas.txt`.

Cosas que no fallaron pero hay que decir.

- **Dónde se corrió.** El harness se ejecutó desde la EC2 con URLs `localhost`, por la razón explicada en el resumen. Para la corrida del profesor, `encender.sh` escribe en `config.yaml` la IP pública que tenga la EC2 ese día y los puertos se abren con el script de calificación.
- **Parte no verificada.** En R11 el reporte avisa que no pudo inspeccionar CloudFront, y no lo contó. Nosotros no usamos CloudFront.

- **Proyección de R12.** El total declarado es US$25,49 y el estimado sobre los recursos en vivo fue US$28,67, calculado con precios bajo demanda; la diferencia (US$3,18) está dentro de la tolerancia de R12.
- **Lambda.** Los 7 servicios facturables de R11 incluyen Lambda, que es del laboratorio y no del proyecto. Los del proyecto son 6.

Cómo reproducirlo. Los pasos están en el repositorio y se ejecutan en AWS, no en local.

```
bash emr/run_emr.sh sample    # raw-sample/ -> curated-sample/
bash emr/run_emr.sh full      # raw/ -> curated/ (sobrescribe)
bash emr/create_tables.sh     # DDL de athena_tables.sql en Glue
pytest                        # harness: report.json y report.md
bash infra/abrir_calificacion.sh    # abre 8000 y 3000 (ventana)
bash infra/cerrar_calificacion.sh   # los cierra
```

7 Trazabilidad con los requisitos

Req.	Decisión que lo sostiene	Sección
R1, R3	EC2 t3.small con API y RDS	3.1
R2	`UPDATE` condicional en PostgreSQL, separación de cargas	3.1, 3.5
R4, R10	EMR efímero con `-auto-terminate`, muestra del 5 % primero	3.2
R5	Glue Data Catalog con *partition projection*	3.3
R6	Parquet y partición por mes	3.3
R7, R9	Endpoint con Athena parametrizado, caché tras la primera consulta	3.4
R8	Frontend estático con nginx en la misma EC2	3.1
R11	Sin NAT, sin balanceador, single-AZ, instancias pequeñas	3.1, 4
R12	`cost_projection.yaml` y su narrativa	5

Tabla 6: De cada requisito a la decisión que lo cubre.

8 Riesgos y mejoras pendientes

- **Sin autenticación en `/admin/flights` ni en el panel.** El contrato lo exige así para el harness. Por eso los puertos solo se abren en la ventana de calificación. En producción irían detrás de HTTPS y autenticación.
- **Single-AZ.** Una caída de la zona deja el sistema fuera. Multi-AZ está prohibido por R11 y lo aceptamos.
- **Caché por proceso.** Cada worker tiene su propio caché y no se invalida cuando EMR vuelve a correr. Con TTL de 10 minutos el desfase es corto. En producción se usaría un caché compartido o invalidación al terminar el job.
- **Rango de *partition projection*.** Está fijo en 2023-01 a 2024-12. Si llegan más meses hay que ampliarlo.
- **Refresco.** La corrida de EMR es manual. Un orquestador programado sería el siguiente paso.

A Bitácora de prompts

Usamos Claude Code como orquestador, que a su vez delegó tareas en agentes (infraestructura, job de EMR, endpoint y frontend, documento). Esta tabla resume los pedidos que dimos y qué decisión provocaron. Son resúmenes, no transcripciones.

N.º	Prompt (resumen)	Efecto en el proyecto
1	Pedir un plan antes de ejecutar nada.	Se escribió `SPEC_P2.md` con infraestructura, capas de S3 y reglas, antes de crear recursos.
2	Usar la semilla 2682 y delegar el trabajo en agentes Sonnet.	Se generaron los datos con esa semilla y el trabajo se repartió: EMR y Athena, endpoint y frontend, documento.
3	No dar nada por bueno sin compararlo con el oráculo.	Se escribió `verificacion_analytics.txt`; conteos, ingresos y top 10 coinciden exactamente.
4	Probar EMR primero con una muestra del 5 % para cuidar el presupuesto.	Primera corrida sobre `raw-sample/`; solo después la corrida completa. Con la corrida de `bookings` el total es de 36 horas normalizadas.
5	Decidir qué hacer con `bookings`, porque el contrato y el oráculo se contradicen.	Se siguió al contrato escrito (solo CONFIRMED) y se declaró aquí.
6	No abrir puertos a internet salvo en la ventana de calificación, porque `/admin/flights` no tiene autenticación.	Scripts `abrir_calificacion.sh` y `cerrar_calificacion.sh`; la base nunca se abre.
7	Redactar este documento corto y directo, por la retroalimentación del parcial 1.	Documento de menos de 20 páginas con cada decisión ligada a un requisito.
8	Rediseñar el frontend con tonos rojos y más trabajado, sin cambiar los selectores del contrato.	Nuevo diseño del cliente y del panel; se volvió a correr el harness (70/70).
9	Probar con otra semilla.	Se aclaró que el profesor usa la misma semilla con otras sondas; se corrieron R4 a R9 con tres *probe salts* distintos y todo pasó.
10	No pagar IP elástica.	Se liberó; `encender.sh` actualiza la IP en `config.yaml` y el costo declarado bajó a US$31,18. Luego se proyectó la EC2 como reservada y quedó en US$25,49.
11	Implementar el *check-in* y las clases del parcial 1 sin romper el contrato.	Clases XS a PR con alias ECONOMY, sillas y *check-in*; el harness siguió en 70/70.

Tabla 7: Bitácora de prompts.